

Practica 6 - Parámetros

1.
 - Parámetro: variable utilizada por un subprograma (función o procedimiento) para recibir o enviar información durante su ejecución.
 - Parámetro real: es el valor, variable o expresión que se envía al invocar una rutina.
 - Parámetro formal: es la variable declarada en la definición del subprograma que recibe el valor del parámetro real (letra o palabra).
 - Ligadura posicional: la asociación entre parámetros reales y formales se realiza según el orden en que aparecen.
 - Ligadura por palabra clave o nombre: la asociación se realiza utilizando el nombre del parametro formal, independientemente del orden. Ejemplo: resta(b=3, a=10) a = 10, b=3 aunque esten invertidos en la llamada.
2.
 - Valor: el parametro real se copia en el parametro formal. Los cambios dentro de la rutina no afectan al original, solo afectan dentro de la subrutina. Ej.:

```
void f(int x) { x = x + 1; print(x); // imprime 6 }  
int a = 5;  
f(a); ← a no cambia  
print(a); // imprime 5
```
 - Valor constante: se pasa por valor pero el parametro no puede modificarse dentro del subprograma. Si se intentara modificarlo, daría error de compilación. Ej.:

```
void f (const int x) { cout << x; } ← solo lectura  
x := x + 1; ← error de compilación
```
 - Resultado: el parametro formal se usa solo para devolver un valor al final del subprograma. Ej.:

```
x = 100; procedure f(out x) { x ++; } → el valor es 1 ya que el valor al entrar al procedimiento es 0 pq se crea una copia local de x
```
 - Nombre: el parametro se comporta como una expresión que se evalúa cada vez que se usa y sirve para devolver valores. Ej.:

```
procedure f(x) x := x + 1;  
llamado por: f(a[i]) ← a[i] se reevalúa cada vez
```
 - Valor-resultado: el parametro entra como copia (valor) y sale con el resultado final. Ej.:

```
z = 1; f(z); procedure f(in out x) x := x + 1; ← ahora z = 2
```

- Referencia: el parametro formal es un alias del real, ambos comparten memoria.
Ej.: void f (int &x) { x = x + 1; }
int a = 5;
f(a); ← a cambia
- Resultado de funciones: es el valor que devuelve una función al finalizar su ejecución mediante return. Es una forma de comunicar información desde la función hacia el programa llamador. Ej.: def suma(a, b) return a + b ;
x = suma (3, 4); ← el resultado de la función es 7

3. a.

Tipo de pasaje de parámetros	Lenguaje
valor / resultado / valor-resultado	Ada
valor / valor constante	C
referencia (objetos)	Ruby
valor (de referencia a objetos)	JAVA
referencia a objetos	Python

b. Ada es mas seguro que Pascal porque, aunque ambos permiten paso por referencia, Ada introduce modos explícitos (in, out, in out) que no solo indican el tipo de pasaje sino tambien restringen formalmente el uso del parametro, permitiendo al compilador detectar errores y evitar modificaciones no deseadas.

c. En ADA, los parámetros **in out** se implementan de distinta manera según el tipo de dato:

1. Tipos escalares (Integer, Float, Boolean, etc): el parametro entra con un valor, se modifica localmente y al terminar se copia al original.
Mecanismo: valor-resultado
2. Tipos compuestos pequeños (records y arreglos pequeños): Ada puede copiar el dato o usar referencia, depende del compilador. Similar a los tipos escalares.
Mecanismo: valor-resultado o referencia
3. Tipos grandes o complejos (arreglos grandes o complejos): la rutina trabaja directamente sobre el objeto original, no conviene copiar porque seria costoso.
Mecanismo: referencia.
4. Tipos con restricciones: Ada verifica restricciones. Para tipos pequeños se usa copia, para tipos grandes, referencia.
Mecanismo: valor-resultado o referencia, dependiendo de tamaño.

5. Tipos protegidos o tareas: nunca se copian. La rutina trabaja sobre el mismo objeto para mantener la integridad, sincronización y concurrencia.

Mecanismo: referencia

4.

c. Si. En el pasaje por resultado (OUT) se produce un error en la línea $m := m + 1 + y$; tanto en cadena estática como dinámica. Ya que el parámetro "y" no está inicializado al entrar a la rutina. Los parámetros por resultado solo se utilizan para devolver información al finalizar la ejecución de la rutina, por lo que leer su valor antes de asignarle uno provoca un error.

d. No. El resultado cambia porque en un lenguaje con cadena dinámica las referencias no locales se resuelven siguiendo la secuencia de llamadas activas. En este caso, la variable m utilizada dentro de Recibe corresponde a la de Dos (quien la llamo) y no a la de Main, como ocurriría con alcance estático.

5. a. Por referencia
b. Por valor en los 3 primeros parámetros, y el último por resultado.

c.

6. El parámetro por nombre evalúa el parámetro real cada vez que el parámetro formal es usado dentro del procedimiento:

1. Parámetro real = valor entero

```
procedure Mostrar( x: integer );  
begin  
    writeln(x);  
end;
```

Mostrar(5); // imprime 5

El valor 5 se usa cada vez que aparece x. Es un valor fijo.

2. Parámetro real = constante

```
const K = 10;  
procedure Doble(x: integer);  
begin
```

```
writeln(x * 2); // 20
end;
```

```
Doble(K);
writeln(K); // imprime 10
```

Cada vez que se usa x se reemplaza por K. La constante no cambia.

3. Parametro real = elemento de un arreglo

```
var
  A: array[1..3] of integer;
  i: integer;

procedure Prueba(x: integer);
begin
  writeln(x);
  i := i + 1;
  writeln(x);
end;
```

```
begin
  A[1] := 100;
  A[2] := 200;
  i := 1;
```

```
  Prueba(A[i]);
end;
```

```
x = A[i]
Primera vez → i = 1 → A[1] = 100
Segunda vez → i = 2 → A[2] = 200
```

El parametro se re-evalua cada vez que se usa, por eso cambia el valor.

4. Parametro real = expresion

```
var
  a, b: integer;

procedure Mostrar(x: integer);
begin
  writeln(x);
  a := a + 1;
```

```
writeln(x);  
end;  
  
begin  
  a := 2;  
  b := 3;  
  
  Mostrar(a + b);  
end;
```

$x = a + b$

Primera vez $\rightarrow 2 + 3 = 5$

Después $a = 3$

Segunda vez $\rightarrow 3 + 3 = 6$

La expresión se recalcula cada vez que se usa el parametro formal.

8. La diferencia entre deep binding y shallow binding aparece cuando se pasan subprogramas (funciones/procedimientos) como parámetros:

- Deep binding (ligadura profunda): el subprograma usa el entorno donde fue pasado como parametro. Recuerda el contexto donde lo envíe
- Shallow binding (ligadura superficial): el subprograma usa el entorno donde se ejecuta. Usa el contexto actual al momento de llamarse.

Cambia el resultado si el subprograma usa variables no locales.